НИЯУ МИФИ

Дисциплина: «Эволюционное программирование
и генетические алгоритмы»

ЛАБОРАТОРНАЯ РАБОТА № 2

Генетический алгоритм для дискретной задачи
с ограничениями

Демонстрационный вариант V=3:
рюкзак с обязательными и взаимоисключающими предметами

Проверяемая реализация на Python · 80 независимых запусков · фактические результаты

2026

Оглавление

1. Вариант и статус постановки 3

2. Цель и постановка задачи 3

2.1. Формальная модель . . . 3

3. Исходные данные и ограничения 3

3.1. Примеры допустимых и недопустимых решений . . . 4

4. Модель и операторы генетического алгоритма 4

4.1. Инициализация и селекция . . . 4

4.2. Кроссовер, мутация, элитизм . . . 4

4.3. Два подхода к ограничениям . . . 4

5. Вычислительная схема и конфигурация 5

5.1. Параметры опытов . . . 6

6. Экспериментальные результаты 6

6.1. Анализ влияния обработки ограничений . . . 6

6.2. Анализ оператора мутации . . . 7

6.3. Сравнение с базовыми методами . . . 7

7. Графики и интерпретация 7

8. Лучшее найденное решение 8

9. Ограничения эксперимента и заключение 9

10. Как воспроизвести результаты 10

Приложение А. Полная таблица объектов 11

Полный список конфликтных пар . . . 12

Приложение Б. Индивидуальные результаты 80 запусков 13

Приложение В. Существенные фрагменты кода 15

Проверка ограничений и их ремонт . . . 15

Бинарная рекомбинация и мутация . . . 15

Точный бюджет оценок . . . 16

Источники и использованные материалы 16

В отчёте использованы только предоставленная методичка (стр. 2–3, 8–12) и фотографии рукописного конспекта (разделы о бинарных хромосомах, селекции, кроссовере, мутации и ограничениях). Внешние источники не использовались.

1. Вариант и статус постановки

Важное уточнение: согласно методичке номер индивидуального варианта вычисляется по N и Q, а для ЛР №2 допустимы только V=1,...,20. Эти N и Q не предоставлены. Число «22» в выражении d=5+(22 mod 6)=9 относится к размерности задачи ЛР №1, а не к определению варианта ЛР №2. Поэтому ЛР №2 выполнена как **демонстрационный V=3**, а не как официально установленный индивидуальный вариант.

Точное название из методички: «Генетический алгоритм для дискретной задачи с ограничениями». Вариант V=3: «Рюкзак с обязательными и взаимоисключающими предметами». Основное представление — бинарная строка с жёсткими ограничениями (методичка, стр. 9).

2. Цель и постановка задачи

Цель из методички: научиться проектировать корректное дискретное представление решения, специальные генетические операторы и методы обработки ограничений на примере задачи выбора, распределения или планирования.

Имеются 40 предметов, для каждого известны масса w_i и полезность v_i. Необходимо выбрать набор максимальной полезности, соблюдая вместимость рюкзака, обязательность некоторых предметов и несовместимость 24 пар предметов. Все данные синтетические и поставляются вместе с кодом.

2.1. Формальная модель

$$x = (x_1, \ldots, x_{40}),\ x_i \in \{0,1\}$$

$$\max F(x) = \sum_i v_i x_i$$

$$\sum_i w_i x_i \le W = 133 \text{ кг}$$

$$x_1 = x_2 = x_3 = 1$$

$$x_a + x_b \le 1 \text{ для каждой конфликтующей пары } (a,b) \in E$$

Бинарное значение $x_i=1$ означает включение предмета в рюкзак, $x_i=0$ — исключение. Генотип — массив из 40 битов; фенотип — выбранный набор предметов. Декодирование заключается в преобразовании единичных позиций в идентификаторы предметов.

3. Исходные данные и ограничения

Данные получены собственным генератором create_problem(seed=2026101802). Масса каждого предмета выбрана равномерно из целых значений 3…20 кг; полезность задана как $v_i=3w_i+r_i$, где r_i — целая псевдослучайная добавка 5…45. Обязательны предметы №1, №2 и №3. Конфликты — 24 различные случайные пары (за исключением пар между обязательными предметами). Вместимость: сумма масс обязательных предметов плюс 87 кг, то есть W=133 кг.

Исключение конфликтов между обязательными объектами обеспечивает существование хотя бы одного допустимого решения. Исходные значения, seed и алгоритм генерации приложены; дополнительные данные из интернета не использовались.

3.1. Примеры допустимых и недопустимых решений

Тип	Номера предметов	Масса	Польз.	Доп.	[m,c,e]
допустимый: только обязательные	1, 2, 3	46 кг	200	Да	[0, 0, 0]
допустимый: жадная эвристика	1, 2, 3, 5, 6, 18, 20, 22, 26, 28, 30, 31, 33, 34, 39	133 кг	811	Да	[0, 0, 0]
недопустимый: нет обязательного	2, 3	26 кг	111	Нет	[1, 0, 0]
недопустимый: конфликт	1, 2, 3, 19	57 кг	270	Нет	[0, 1, 0]
недопустимый: перегруз и конфликты	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17…	441 кг	2263	Нет	[0, 24, 308]

В последнем столбце указаны: m — число пропущенных обязательных предметов, c — число конфликтующих выбранных пар, e — превышение вместимости в килограммах. Пример «нет обязательного» нарушает $x_1=1$; пример «конфликт» содержит несовместимые предметы №1 и №19; набор всех 40 предметов одновременно нарушает массу и конфликты.

4. Модель и операторы генетического алгоритма

4.1. Инициализация и селекция

Каждая из 48 стартовых хромосом формируется независимым включением каждого предмета с вероятностью 0,32. В режиме «ремонт» каждая хромосома сразу исправляется; в режиме «штраф» допускается присутствие недопустимых хромосом, а качество регулируется штрафной оценкой.

Для выбора родителей используется турнир размером k=3: из текущей популяции с возвращением случайно выбираются три кандидата, родителем становится кандидат с максимальным значением используемого фитнеса. В качестве альтернативы рассмотрен в конспекте рулеточный отбор, однако выбран турнирный, потому что он корректно работает и для отрицательных штрафных оценок и не требует масштабирования.

4.2. Кроссовер, мутация, элитизм

Рекомбинация — равномерный кроссовер с вероятностью $P_c=0{,}88$: для каждого бита выбирается ген одного из родителей, что позволяет комбинировать отдельные включённые предметы. Если кроссовер не применяется, потомки копируют родителей.

Сравниваются два оператора мутации, при неизменной структуре алгоритма: (А) побитовая инверсия с вероятностью $p=1/40$ на каждом гене; (Б) обмен двух случайных позиций, выполняемый один раз на потомке. Обмен сохраняет количество единиц до коррекции ограничений; побитовая мутация может изменять это количество.

Элитизм: в каждом поколении две наиболее приспособленные хромосомы переносятся без изменений. Однако глобально лучшее **допустимое** решение запоминается отдельно — «best-ever», в том числе при использовании штрафов.

4.3. Два подхода к ограничениям

Ремонт. В хромосому принудительно включаются обязательные предметы; затем из несовместимой пары исключается необязательный предмет с меньшей полезностью на килограмм; далее, пока превышена масса, удаляется необязательный выбранный предмет с минимальным отношением полезности к массе. После такого ремонта полученная хромосома удовлетворяет всем ограничениям (при условии выполнимого обязательного набора).

Штрафы. Исходная полезность корректируется штрафами за каждое нарушение. Это не гарантирует допустимость промежуточного поколения, поэтому итоговым ответом считается только лучшее допустимое решение, реально встретившееся во всех поколениях.

```
Fitness(x) = F(x) − 400m(x) − 250c(x) − 25e(x)
```

Где m — число пропущенных обязательных объектов, c — число выбранных конфликтных пар, e — превышение веса (кг). Коэффициенты — явно зафиксированные параметры эксперимента, а не теоретически оптимальные константы.

Как отмечено в рукописном конспекте (лекция 4, стр. 26–30), бинарное кодирование удобно для задач выбора, но кроссовер и мутация могут порождать недопустимые хромосомы. Поэтому процедуры восстановления допустимости и явного учёта ограничений — обязательные части реализации.

5. Вычислительная схема и конфигурация

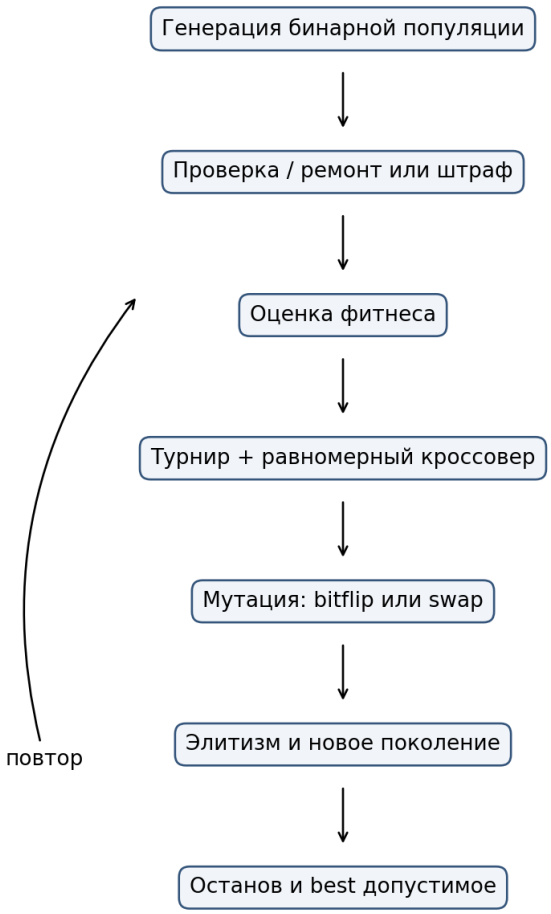


Рисунок 1 — собственная схема цикла генетического алгоритма.

5.1. Параметры опытов

Параметр	Значение
Число объектов	40
Размер популяции	48
Поколения	120
Число элитных особей	2
Турнир	3
Pc	0.88
p_mut	1/40 на ген
Тип кроссовера	Равномерный
Начальная вероятность бита 1	0.32
Критерий останова	120 поколений
Число независимых запусков на вариант	20
Общий бюджет оценок на запуск	5522
Семена запусков	2026102000–2026102019
Seed данных	2026101802

Оценивались 48 стартовых особей и 46 новых потомков в каждом из следующих 119 поколений: B = 48 + 119·46 = 5 522. Элитные особи используют сохранённые значения фитнеса и не переоцениваются. Для случайного поиска сформировано ровно B новых кандидатов.

Фиксированные seed позволяют воспроизвести результаты. Все четыре конфигурации использовали одинаковый список из 20 seed. Сравнения попарно изменяли ровно один фактор: «ремонт/bitflip» vs «штраф/bitflip» — только обработка ограничений; «ремонт/bitflip» vs «ремонт/swap» — только мутация.

6. Экспериментальные результаты

Конфигурация	Лучшее	Среднее	Медиана	σ	Худшее	Допуст.
ГА: ремонт / bitflip	811	810.15	811.0	3.80	794	100.0%
ГА: штраф / bitflip	811	794.65	796.0	11.49	774	39.7%
ГА: ремонт / swap	811	807.05	811.0	7.06	794	100.0%
Случайный допустимый	811	788.50	786.5	8.98	776	100.0%

Показаны распределения лучших допустимых значений 20 независимых запусков каждого метода. Стандартное отклонение является выборочным (ddof=1). Для решения о качестве важны и медиана, и худший результат: одиночный удачный запуск не гарантирует устойчивости.

6.1. Анализ влияния обработки ограничений

При побитовой мутации ремонт дал среднее 810,15 и медиану 811, тогда как штрафной подход — среднее 794,65 и медиану 796. Доля допустимых оцененных хромосом: 100% против примерно 39,7%. Прямой ремонт оказал положительное влияние в этой конкретной синтетической задаче, но он также уменьшает исследуемое пространство и может мешать в других задачах.

6.2. Анализ оператора мутации

При одинаковой процедуре ремонта побитовая мутация дала среднее 810,15 против 807,05 для обмена двух позиций. Это согласуется с тем, что простой обмен сохраняет число выбранных предметов и может хуже исследовать комбинации разной мощности; после ремонта это свойство частично нарушается. Вывод относится к выбранным настройкам и синтетическим данным.

6.3. Сравнение с базовыми методами

Случайный допустимый поиск при том же бюджете показал среднее 788,50 и медиану 786,5 — хуже рассматриваемых ГА, но также нашёл лучший результат 811 хотя бы в одном запуске. Отдельная жадная эвристика по отношению полезность/масса также достигла 811. Следовательно, для данного экземпляра **не доказано преимущество ГА над конструктивной жадной эвристикой**; сравнение с ней приведено без искусственного завышения результата ГА.

7. Графики и интерпретация

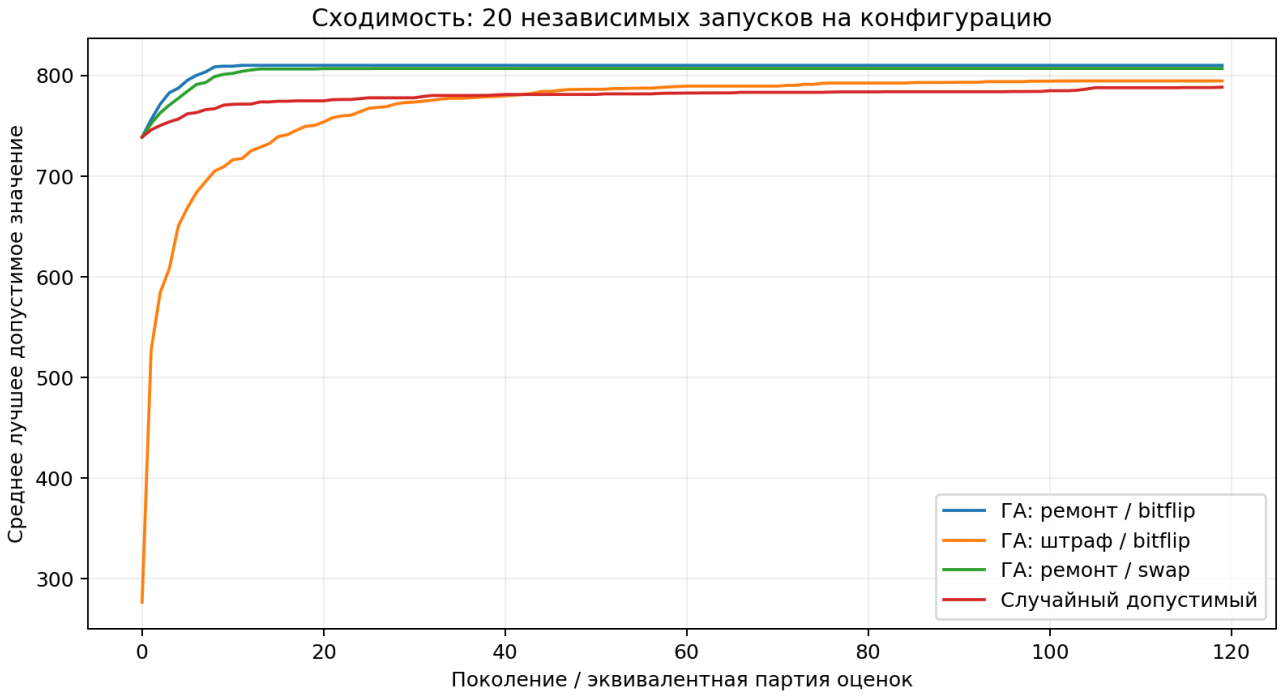


Рисунок 2 — средние траектории лучшего допустимого значения по поколениям.



Рисунок 3 — разброс итоговых результатов по 20 запускам.

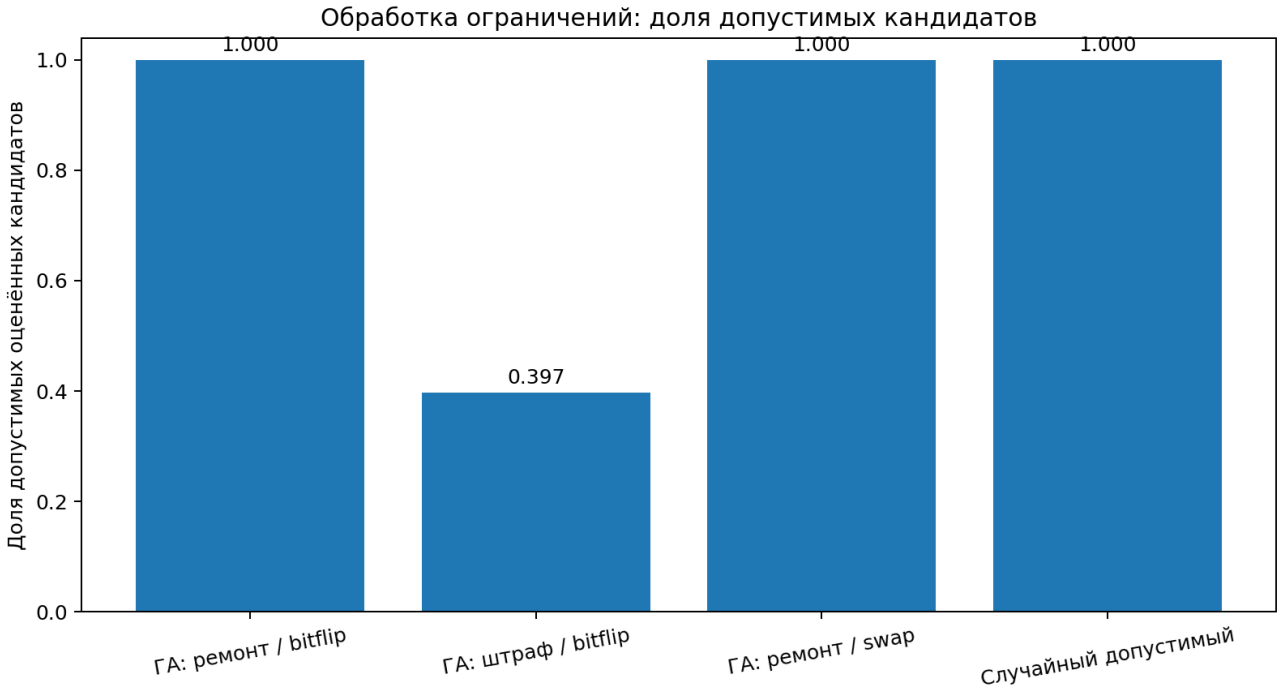


Рисунок 4 — сравнение доли допустимых кандидатов.

При использовании штрафов недопустимые особи могут сохраняться в популяции и влиять на отбор. Однако графики измеряют только качество лучшего **допустимого** найденного решения; смешивать его со штрафным фитнесом некорректно. Для случайного поиска «поколение» обозначает эквивалентную партию оценок, а не эволюционный шаг.

8. Лучшее найденное решение

Лучший найденный допустимый комплект содержит 15 предметов: 1, 2, 3, 5, 6, 18, 20, 22, 26, 28, 30, 31, 33, 34, 39.

Полезность = **811**, масса = **133 кг** из 133 кг, seed = **2026102000**, конфигурация: ГА: ремонт / bitflip.

№ предмета	Масса, кг	Полезность	Обязательный
1	20	89	Да
2	7	49	Да
3	19	62	Да
5	5	58	Нет
6	3	28	Нет
18	3	44	Нет
20	13	70	Нет
22	3	40	Нет
26	13	72	Нет
28	6	36	Нет
30	5	45	Нет
31	15	90	Нет
33	4	28	Нет
34	5	32	Нет
39	12	68	Нет

Контроль допустимости лучшего решения: отсутствующие обязательные предметы — 0; конфликтные выбранные пары — 0; превышение вместимости — 0. Проверки выполнены программно и сохранены в summary.json.

Значение 811 — лучшее из наблюдавшихся при указанном конечном бюджете, **не доказанный глобальный оптимум**. Для строгого доказательства оптимальности необходим исчерпывающий перебор с отсечениями либо точный алгоритм (в задаче с 40 двоичными переменными это потенциально существенно дороже).

9. Ограничения эксперимента и заключение

Задача достаточно дискретна и нелинейна по допустимому множеству: обязательные предметы и конфликты запрещают многие сочетания. Бинарная популяция обоснованна, поскольку единичный бит имеет прямой предметный смысл. Равномерное скрещивание и две мутации специально работают с бинарной хромосомой; их результат может быть недопустим, что продемонстрировано и обрабатывается двумя исследованными методами.

Качество ГА «ремонт / bitflip» в данном эксперименте хорошее относительно равнобюджетного случайного поиска, прежде всего из-за высокой устойчивости. Но так как жадная эвристика тоже находит 811, нельзя делать вывод, что эволюционный поиск необходим для этого конкретного экземпляра. Дополнительная проверка на разных экземплярах и более сложных конфликтах улучшила бы обобщаемость результатов.

Вычислительный бюджет измеряется числом вызовов исходной функции полезности; стоимость самого ремонта в этот бюджет не входит. Поэтому время исполнения алгоритмов не обязано быть одинаковым. Коэффициенты штрафов и частоты мутаций эмпирические и могут потребовать перенастройки на других данных.

Результаты ограничены 20 seed и одним синтетическим набором из 40 предметов. Статистическое сравнение не заменяет строгого доказательства оптимальности. Код проверен автоматическими

тестами (12 тестов).

10. Как воспроизвести результаты

Пример запуска из каталога лабораторной работы:

```
python -m pip install -r requirements.txt
python main.py --out results --runs 20 --seed-base 2026102000
python -m unittest -v test_lab2.py
```

Устанавливать готовые библиотеки эволюционной оптимизации не требуется; генетические операторы реализованы вручную. В архив включены main.py, test_lab2.py, README.md, requirements.txt, requirements-lock.txt, файл генерации PDF, входные данные, таблицы, 4 диаграммы и протокол тестов.

Независимые запуски: runs.csv содержит seed, лучшую допустимую полезность, массу, число оценок и выбранные предметы; trajectories.csv — значения по поколениям. Параметры сохранены в config.json, данные — в items.csv, conflicts.csv, instance.json.

Приложение А. Полная таблица объектов

Предмет	Масса, кг	Полезность	Обязательный
1	20	89	Да
2	7	49	Да
3	19	62	Да
4	11	67	Нет
5	5	58	Нет
6	3	28	Нет
7	16	84	Нет
8	6	43	Нет
9	9	41	Нет
10	9	49	Нет
11	10	44	Нет
12	18	78	Нет
13	14	71	Нет
14	20	91	Нет
15	14	48	Нет
16	17	61	Нет
17	7	47	Нет
18	3	44	Нет
19	11	70	Нет
20	13	70	Нет
21	5	27	Нет
22	3	40	Нет
23	17	88	Нет
24	14	49	Нет
25	16	78	Нет
26	13	72	Нет
27	12	63	Нет
28	6	36	Нет
29	12	54	Нет
30	5	45	Нет
31	15	90	Нет
32	7	33	Нет
33	4	28	Нет
34	5	32	Нет

Предмет	Масса, кг	Полезность	Обязательный
35	15	79	Нет
36	15	53	Нет
37	16	53	Нет
38	8	41	Нет
39	12	68	Нет
40	9	40	Нет

Полный список конфликтных пар

Пара 1	Пара 2	Пара 3	Пара 4
1–19	2–17	3–9	4–5
4–9	4–12	4–21	4–24
5–8	5–21	6–7	7–8
7–36	10–18	13–32	14–21
17–18	19–34	19–36	23–25
25–32	27–38	32–38	37–38

Приложение Б. Индивидуальные результаты 80 запусков

Алгоритм	Seed	Лучшая польз.	Масса	Доля доп.	Оценок
R/B	2026102000	811	133	1	5522
R/B	2026102001	811	133	1	5522
R/B	2026102002	811	133	1	5522
R/B	2026102003	811	133	1	5522
R/B	2026102004	811	133	1	5522
R/B	2026102005	811	133	1	5522
R/B	2026102006	811	133	1	5522
R/B	2026102007	811	133	1	5522
R/B	2026102008	811	133	1	5522
R/B	2026102009	811	133	1	5522
R/B	2026102010	811	133	1	5522
R/B	2026102011	811	133	1	5522
R/B	2026102012	811	133	1	5522
R/B	2026102013	811	133	1	5522
R/B	2026102014	811	133	1	5522
R/B	2026102015	811	133	1	5522
R/B	2026102016	794	133	1	5522
R/B	2026102017	811	133	1	5522
R/B	2026102018	811	133	1	5522
R/B	2026102019	811	133	1	5522
P/B	2026102000	804	132	0.38572	5522
P/B	2026102001	802	133	0.38247	5522
P/B	2026102002	780	133	0.41216	5522
P/B	2026102003	781	133	0.38953	5522
P/B	2026102004	795	133	0.40492	5522
P/B	2026102005	797	133	0.38174	5522
P/B	2026102006	793	132	0.42031	5522
P/B	2026102007	811	133	0.41072	5522
P/B	2026102008	800	133	0.38917	5522
P/B	2026102009	802	133	0.39985	5522
P/B	2026102010	811	133	0.40329	5522
P/B	2026102011	789	133	0.40764	5522
P/B	2026102012	779	133	0.40456	5522

Алгоритм	Seed	Лучшая польз.	Масса	Доля доп.	Оценок
P/B	2026102013	777	133	0.38518	5522
P/B	2026102014	774	133	0.39152	5522
P/B	2026102015	802	133	0.41053	5522
P/B	2026102016	794	133	0.39750	5522
P/B	2026102017	797	133	0.38808	5522
P/B	2026102018	811	133	0.40293	5522
P/B	2026102019	794	133	0.36888	5522
R/S	2026102000	794	133	1	5522
R/S	2026102001	811	133	1	5522
R/S	2026102002	811	133	1	5522
R/S	2026102003	794	133	1	5522
R/S	2026102004	794	133	1	5522
R/S	2026102005	811	133	1	5522
R/S	2026102006	811	133	1	5522
R/S	2026102007	811	133	1	5522
R/S	2026102008	797	133	1	5522
R/S	2026102009	811	133	1	5522
R/S	2026102010	811	133	1	5522
R/S	2026102011	811	133	1	5522
R/S	2026102012	811	133	1	5522
R/S	2026102013	811	133	1	5522
R/S	2026102014	811	133	1	5522
R/S	2026102015	811	133	1	5522
R/S	2026102016	797	133	1	5522
R/S	2026102017	811	133	1	5522
R/S	2026102018	811	133	1	5522
R/S	2026102019	811	133	1	5522
RAND	2026102000	802	132	1	5522
RAND	2026102001	795	133	1	5522
RAND	2026102002	786	133	1	5522
RAND	2026102003	782	132	1	5522
RAND	2026102004	776	129	1	5522
RAND	2026102005	794	133	1	5522
RAND	2026102006	779	132	1	5522
RAND	2026102007	777	133	1	5522
RAND	2026102008	786	133	1	5522

Алгоритм	Seed	Лучшая польз.	Масса	Доля доп.	Оценок
RAND	2026102009	790	131	1	5522
RAND	2026102010	782	133	1	5522
RAND	2026102011	795	133	1	5522
RAND	2026102012	786	133	1	5522
RAND	2026102013	793	132	1	5522
RAND	2026102014	811	133	1	5522
RAND	2026102015	787	131	1	5522
RAND	2026102016	784	133	1	5522
RAND	2026102017	779	133	1	5522
RAND	2026102018	787	133	1	5522
RAND	2026102019	799	133	1	5522

Обозначения: R/B — repair + bitflip; P/B — penalty + bitflip; R/S — repair + swap; RAND — случайный допустимый поиск. Полные траектории и хромосомы — в машиночитаемых CSV архива.

Приложение В. Существенные фрагменты кода

Ниже приводятся основные операции из запускаемой программы. Полные исходники доступны в ZIP и являются частью сдаваемого результата.

Проверка ограничений и их ремонт

```
def violation(x, p):
    missing = sum(1 for i in p.mandatory if not x[i])
    conflicts = sum(1 for i, j in p.conflicts
                    if x[i] and x[j])
    excess = max(0, used_weight(x, p) - p.capacity)
    return missing, conflicts, excess

# Полный repair() см. в main.py.
```

Бинарная рекомбинация и мутация

```
def crossover(a, b, rng, p_cross):
    if rng.random() >= p_cross:
        return a.copy(), b.copy()
    mask = rng.random(len(a)) < .5
    return (np.where(mask, a, b).copy(),
            np.where(mask, b, a).copy())

def mutate(x, rng, mode, p_gene):
    y = x.copy()
    if mode == 'bitflip':
        mask = rng.random(len(y)) < p_gene
        y[mask] = ~y[mask]
    elif mode == 'swap':
        if rng.random() < min(1.0, p_gene * len(y)):
            i, j = rng.choice(len(y), size=2, replace=False)
            y[i], y[j] = y[j], y[i]
    return y
```

Точный бюджет оценок

```
@property
 def budget(self) -> int:
     return self.population + (
         self.generations - 1
     ) * (self.population - self.elite)

# В данной конфигурации: 48 + 119 * 46 = 5522.
```

Источники и использованные материалы

1. Душкин Р. В. «Эволюционное программирование и генетические алгоритмы». Комплекс лабораторных работ. Стр. 2–3, 8–12 (источник постановки, минимальных масштабов, требований к экспериментам и операторов).

2. Фотографии рукописных конспектов: лекция 1–2 (базовый эволюционный цикл и статистика), лекция 4, стр. 25–30 (бинарное кодирование, селекция, кроссовер, мутация, обработка ограничений).

3. Собственные синтетические данные, Python-код, тесты и результаты запусков; seed и структура генератора указаны выше. Внешние интернет-источники не привлекались.